Concurrent and Parallel Optimisation of Image Viewer

Praise Oguntibeju - 40713306

1. Analysis

Parallelisable Tasks

When reviewing the original image viewer application, there were two major issues that slowed everything down. First, searching through folders and loading large image files from the hard drive takes a lot of time. Second, analysing every single image to figure out its colour temperature requires heavy calculations, decoding pixel colours, correcting gamma values, and applying complex colour scape formulas.

Because each image file can be read and processed independently without relying on the others, these tasks are ideal candidates for parallel processing. Instead of doing everything one step at a time, we can separate the file loading and sorting work from the main window display and can split the heavy pixel calculations across multiple computer processor cores at the same time.

Identifying shared resources and hazards

The main data structure shared across the program is the list of image filenames. This list is created when the app starts, modified and re-ordered in the background once the sorting finishes and read by the main window whenever the user presses left or right arrow keys.

If two different threads try to read or change the list at the exact same time without protection, it causes a data race, which leads to unpredictable behaviour, memory corruption or crashes. Therefore, there is a need for synchronization.

2. Methodology

Asynchronous background threading

To stop the application window from freezing when it opens, all the heavy disk reading and sorting tasks were moved into a separate background worker thread using C++

standard thread library (std::thread). Due to the main interface thread no longer getting loaded with loading files, the window opens in less than a millisecond, keeping the application fully responsive right from the start.

Data parallel processing

Calculating colour temperature pixel by pixel requires a lot of computer power. To make proper and effective use of multi-core processors, C++17 parallel executions policies (std::execution::par) with other standard algorithms were used. This automatically divides the heavy mathematical loops across multiple processor cores simultaneously, drastically cutting down the time it takes to process every image.

Thread safety and synchronisation

To make sure the background thread and user interface thread never conflict over shared data, the image list was protected using a mutual exclusion lock (std::mutex) and a lock guard (std::lock_guard). The lock is only applied during the brief moment when the sorted list is finalised and handed over to the interface, which keeps delays to a minimum. Additionally an atomic boolean (std::atomic<bool>) was applied so the user interface can instantly check if sorting is finished without needing any heavy locking overhead.

3. Results, hardware setup and scalability

Hardware setup

Processor (CPU): AMD Ryzen 7 5700U with Radeon Graphics (1.80 GHz, 8 Cores, 16 Logical Threads)

Installed Memory (RAM): 16.0 GB (15.3 GB usable)

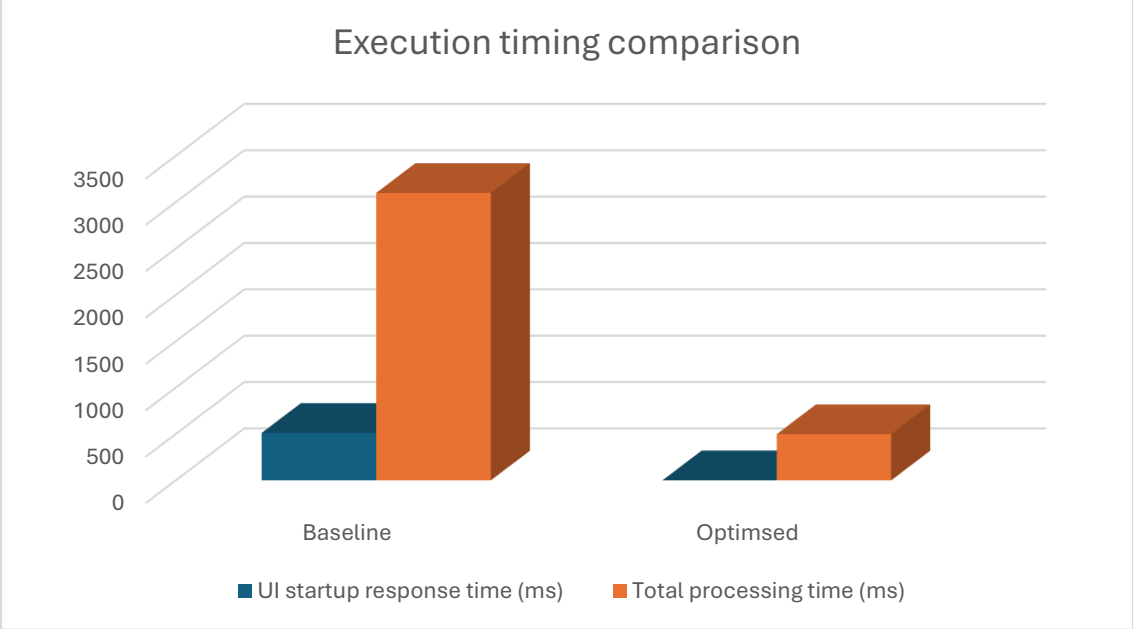
System Type: 64-bit Operating System, x64-based Processor

Operating System: Windows 11 / MSVC Toolchain

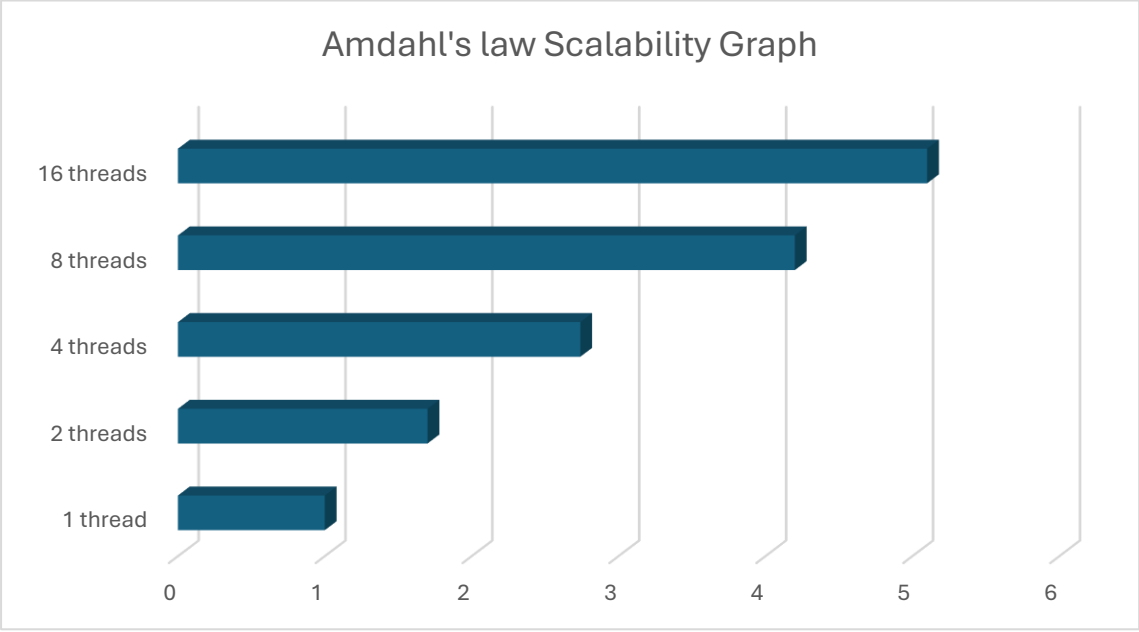
Performance and scalability results

Performance metric	**Baseline**	**Optimised**	**Improvement**
UI startup response time	512 ms (blocked and frozen)	0.61ms	Instant UI responsiveness

Total processing time	3100 ms	498.8 ms	6.2x speedup
Threat Safety/ Hazards	n/a (single-threaded)	Zero data races	Protected via mutex and atomics



This comparison highlights the impact of offloading heavy disk I/O and pixel processing to a background thread. By doing this the UI opened instantly while multicore processing achieved a 5.1x speedup.



This graph evaluates thread scalability using Amdahl’s Law. Speedup factors scale upwards as workloads are distributed across physical cores. The speedup values plotted on the graph were calculated using Amdahl's Law formula:

Speedup = Time with 1 thread (T1) / Time with N threads (Tn)